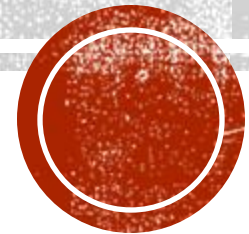


WEEK 7

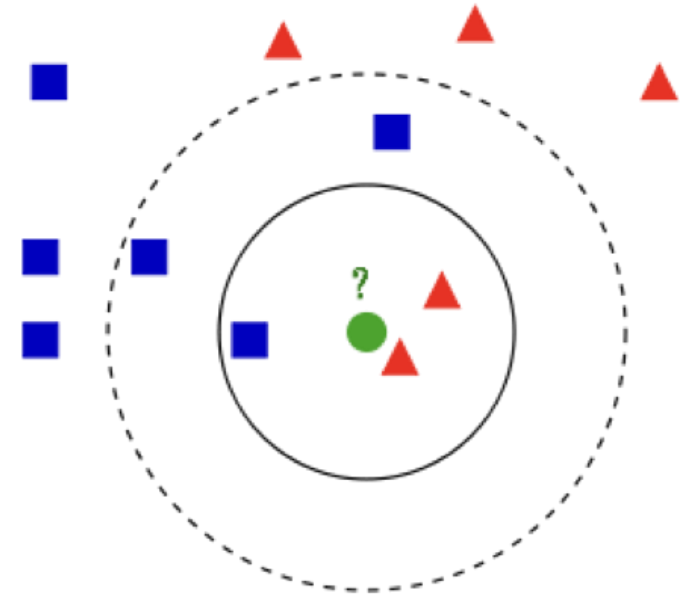
Non-Parametric Method



K-NEAREST NEIGHBOUR

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- Simple classification technique
- Label the data based on the neighbours
- K= represents number of neighbours



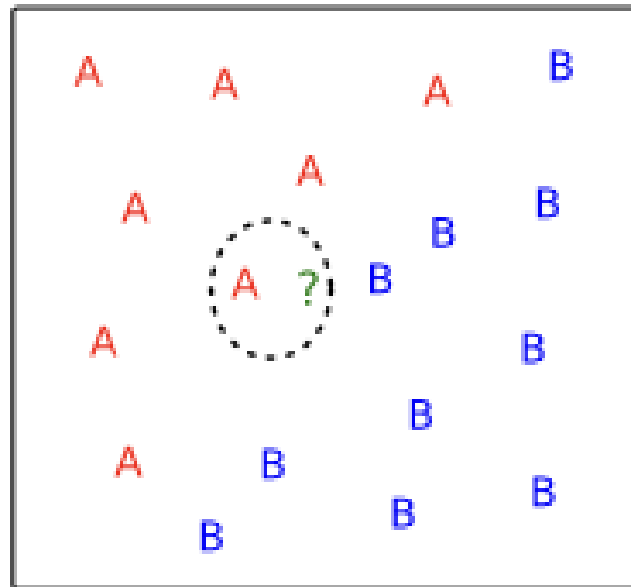
ALGORITHM

- Unweighted: = All features are the same
- Distance-weighted: $\hat{y} = \operatorname{argmax}_c \sum_i w_i 1[y_i = c]$, with $w_i = 1 / (d(\mathbf{x}, \mathbf{x}_i) + \varepsilon)$
- - Regression: average or weighted average of neighbor targets.
- Key levers:
 - - Distance metric (Euclidean, Manhattan, Minkowski, cosine).
 - - Feature scaling matters (put StandardScaler in a Pipeline).
 - - Choice of k trades variance (small k) vs bias (large k).

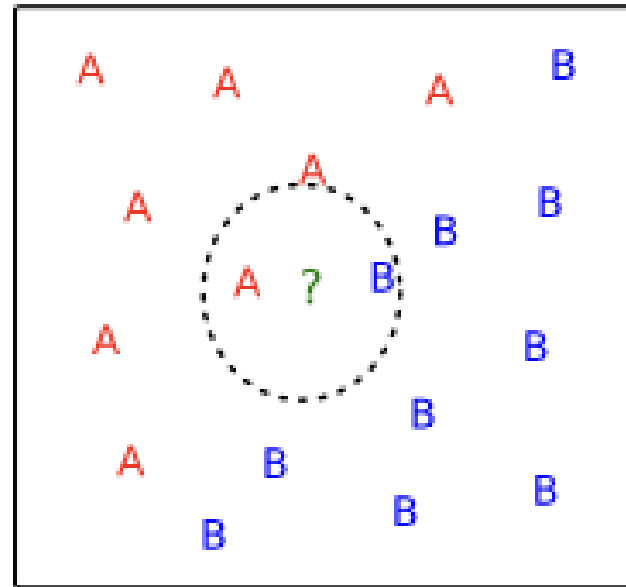


DIFFERENT K

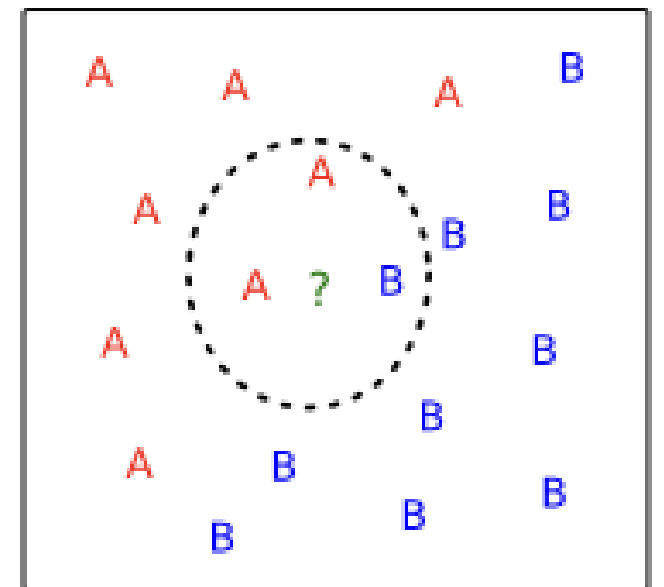
1st, 2nd, and 3rd Nearest Neighbors
of a Test Instance



1-nearest neighbor



2-nearest neighbor



3-nearest neighbor

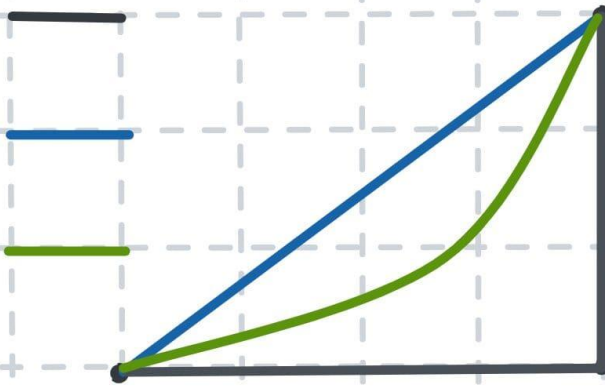


DISTANCE

Manhattan

Euclidean

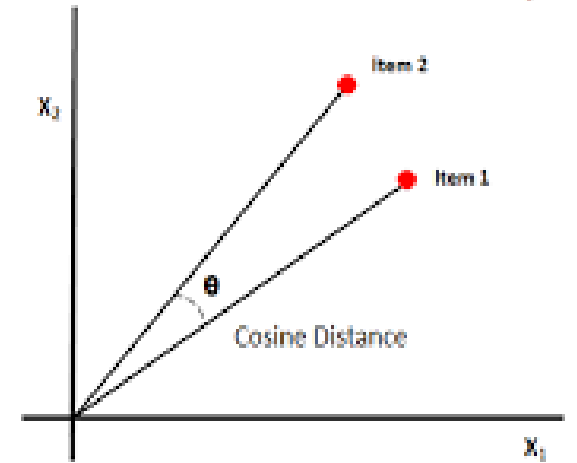
Minkowski,
for p



Minkowski when $p = 1$ --> Manhattan
Minkowski when $p = 2$ --> Euclidean

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p} \text{ for } p \geq 1$$

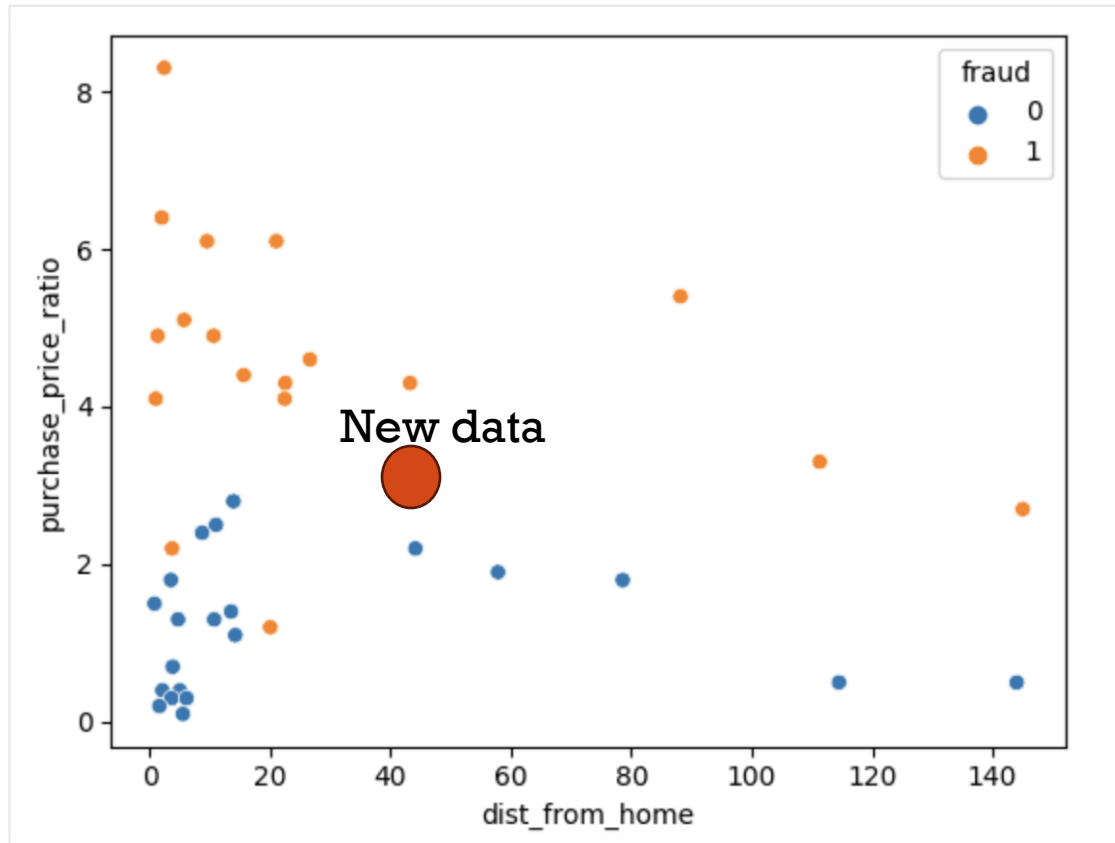
Cosine Distance/Similarity



$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$



EXAMPLE:

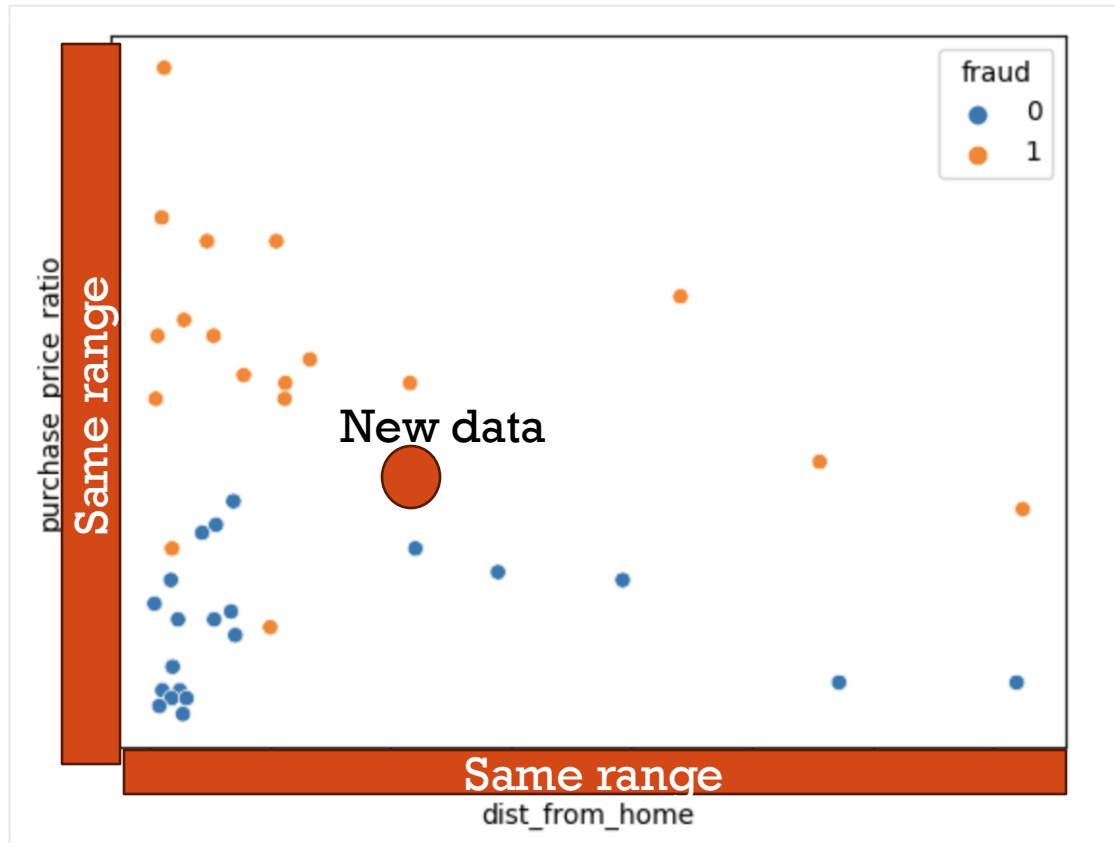


Be careful!!

KNN is very sensitive to scaling

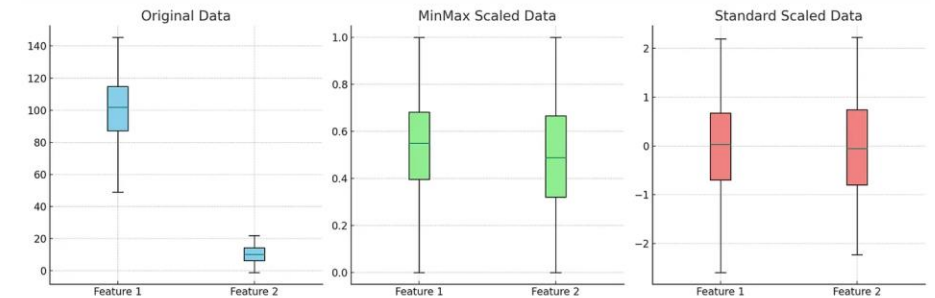


DATA SCALING IS IMPORTANT



Be careful!!

KNN is very sensitive to scaling



EXAMPLE DATA

X	y	Group
1	2	A
4	4	A
2	4	B
3	3	B



EXAMPLE DATA

NEW DATA
(3,5)

X	y	Group	EUCLIDEAN	Manhattan
1	2	A	$\sqrt{13}$	5
4	4	A	$\sqrt{2}$	2
2	4	B	$\sqrt{2}$	2
3	3	B	$\sqrt{4}$	2

K=1

K=3



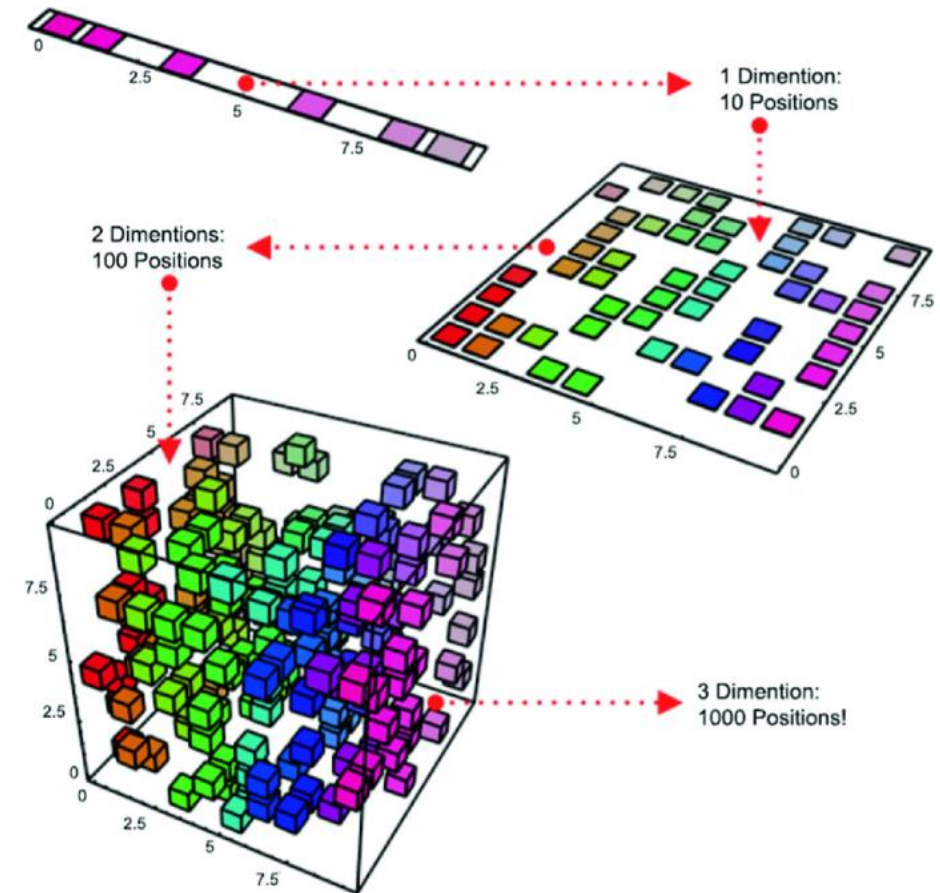
PROBLEM WITH SCALING!!

X	y	Group
0.1	20	A
0.4	49	A
0.2	114	B
0.3	30	B



CURSE OF DIMENSIONALITY

- High dimensionality makes curse of dimensionality
- Use DIM REDUCTION → KNN to find the output



PIPELINE WITH KNN

- `pipeline = make_pipeline(StandardScaler(),`

`PCA(),`

`KNeighborsClassifier(n_neighbors=3))`



CODE EXAMPLE 1

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold

pipe_knn = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())
])

param_grid_knn = {
    "knn__n_neighbors": [3, 5, 7, 11],
    "knn__weights": ["uniform", "distance"],
    "knn__metric": ["euclidean", "manhattan"]
}
```



CODE EXAMPLE 2

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

gs_knn = GridSearchCV(pipe_knn, param_grid_knn, cv=cv, n_jobs=-1,
scoring="accuracy")

gs_knn.fit(X_train, y_train)


print(gs_knn.best_params_, gs_knn.best_score_)

# Final one-time evaluation:

print("Test acc:", gs_knn.score(X_test, y_test))
```



QUESTIONS

- How to choose **K**?
- Why it can be slow?
- Why other methods are faster?
- Non parametric?

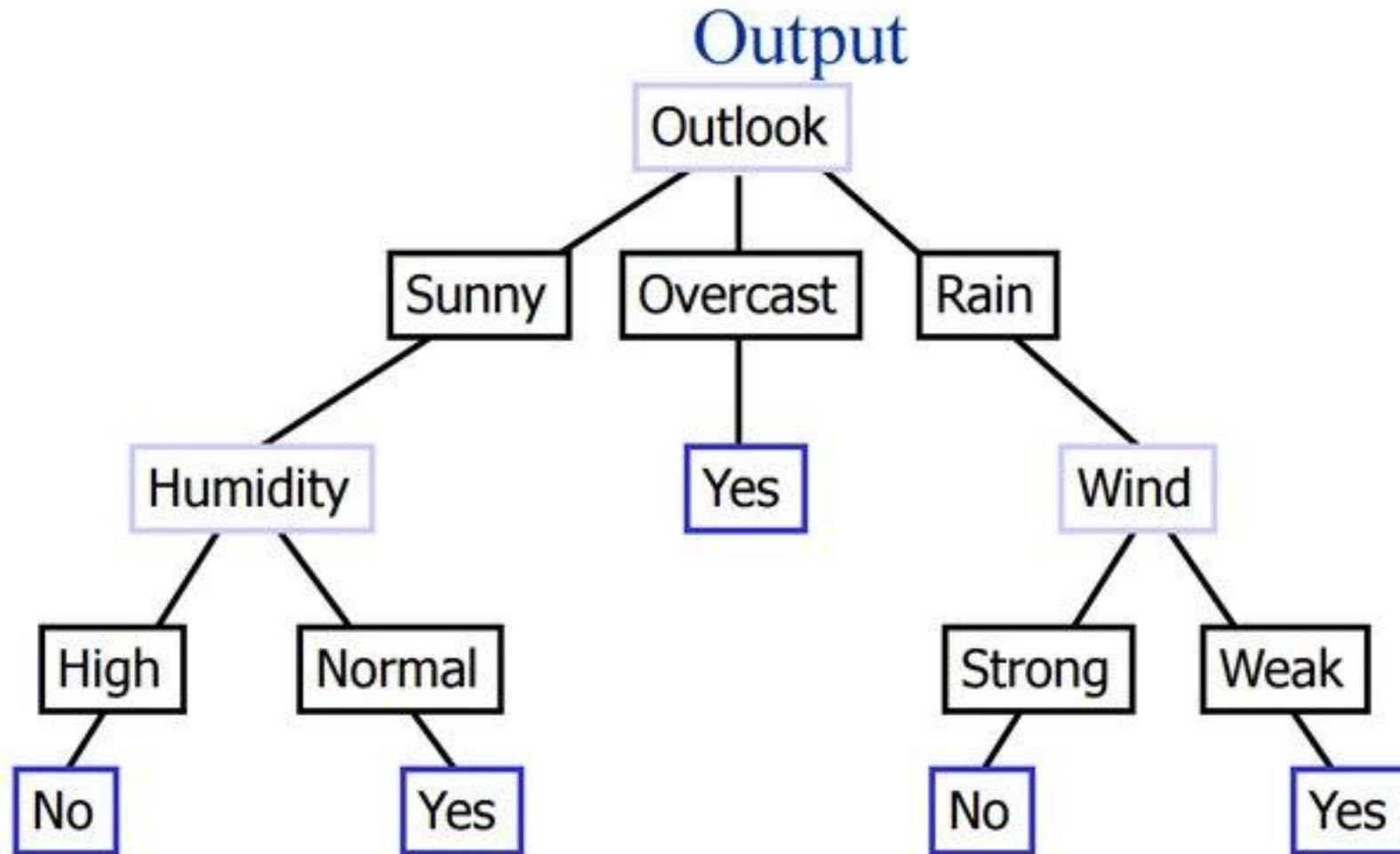


KNN SUMMARY

- No actual separate training phase
- Can generate complex decision boundaries
- Can be slow
 - Distance between new sample and all samples must be computed to classify new sample

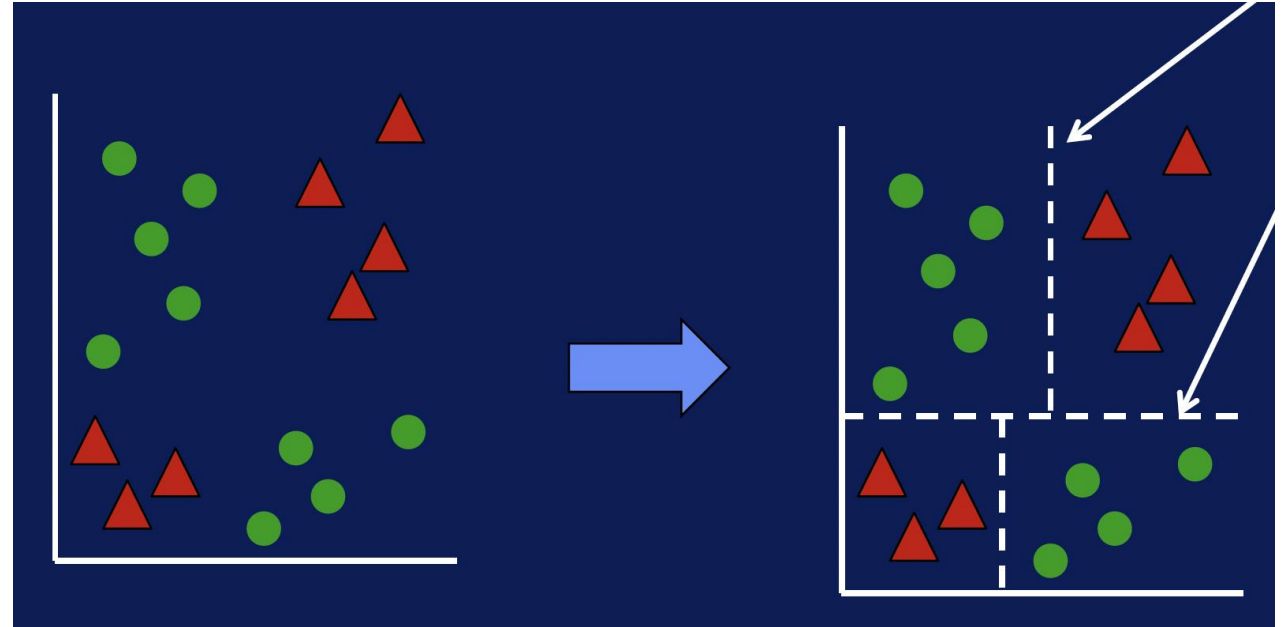


DECISION TREE



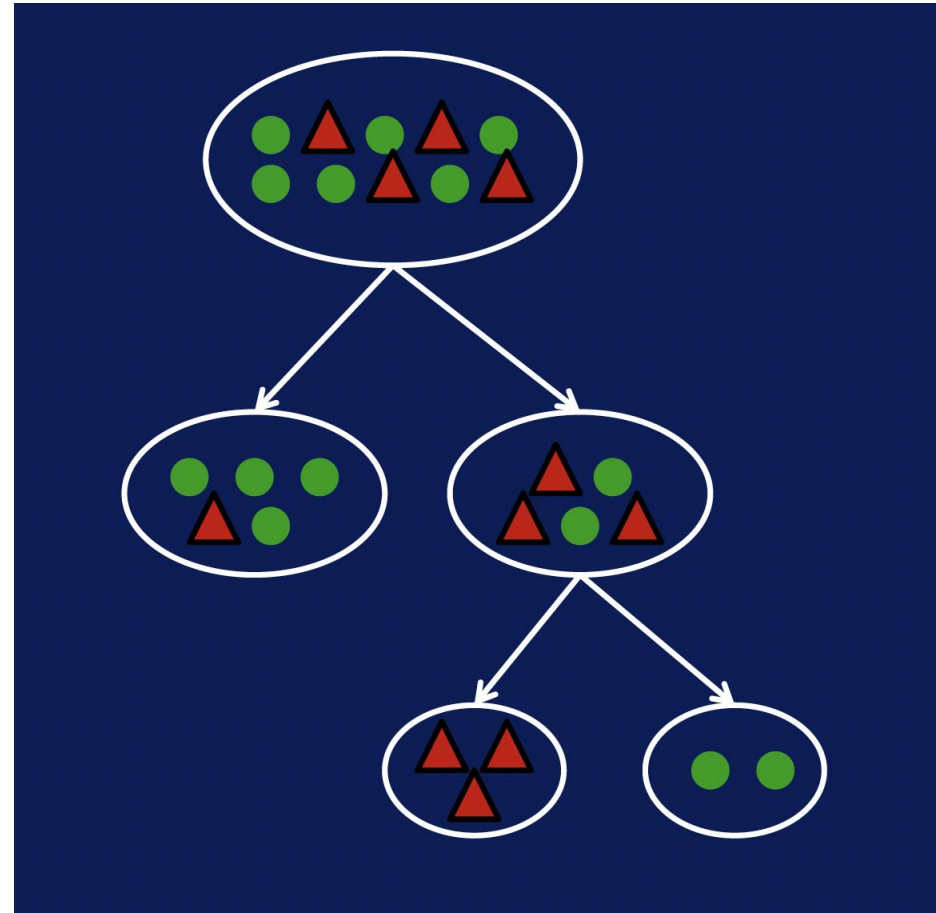
CONSTRUCTING DECISION TREE

- Start with all samples at a node.
- Partition samples based on input to create purest subsets.
- Repeat to partition data into successively purer subsets.



FINALIZED DECISION TREE

- Test split on all variables
- Resulting tree is often simple and easy to interpret
- Induction is computationally inexpensive
- The approach does not guarantee best solution
- Rectilinear decision boundaries



IMPURITY MEASURES (CLASSIFICATION):

- Gini:
$$gini(T) = 1 - \sum_{j=1}^n p_j^2$$
 :Probability of other classes

- Entropy: $H(S)$
$$entropy(T) = - \sum_{j=1}^n p_j \cdot \log p_j$$

- Information gain for split A:

$$Gain_{split}(T) = entropy(T) - \frac{N_1}{N} \cdot entropy(T_1) - \frac{N_2}{N} \cdot Entropy(T_2)$$

- Hyperparameters that control complexity:
 - max_depth
 - min_samples_leaf
 - min_samples_split
 - max_features



BENEFITS AND DRAWSBACK

- Deep trees → low bias, high variance (overfit).
- Shallow trees → higher bias, lower variance (underfit).
- Use cross-validation to pick `max_depth` and `min_samples_leaf`.
- Interpretability:
 - Feature importance and path explanations are easy to extract.
 - Fast inference



EXAMPLES

```
from sklearn.tree import DecisionTreeClassifier

pipe_tree = Pipeline([
    ("scaler", StandardScaler(with_mean=True, with_std=True)), # not required for trees, but harmless
    ("tree", DecisionTreeClassifier(random_state=0))
])

param_grid_tree = {
    "tree__max_depth": [None, 3, 5, 8],
    "tree__min_samples_leaf": [1, 3, 5],
    "tree__criterion": ["gini", "entropy"]
}
```



CODE EXAMPLE 2

```
gs_tree = GridSearchCV(pipe_tree, param_grid_tree, cv=cv, n_jobs=-1,  
scoring="accuracy")  
  
gs_tree.fit(X_train, y_train)  
  
print(gs_tree.best_params_, gs_tree.best_score_)  
  
print("Test acc:", gs_tree.score(X_test, y_test))
```



EXAMPLE RESULT

Decision tree trained on all the iris features

